

SSN College of Engineering

LABORATORY RECORD

Subject: Web Application Development Laboratory

Exercise 10: DatabaseServlet Application using Tomcat Server

SUBMITTED BY

Name: Vijayaraaghavan K S

Register Number: 3122247001066

Branch: Department of CSE

Semester / Year: V Semester / III Year

Subject Code: ICS1511 Batch: 2024–2029

1. Problem Description

Scenario

`index.html` presents a form with a submit button whose `method="get"` attribute sends a GET request to a Java Servlet (`DatabaseServlet`). The servlet connects to a MySQL database via JDBC, runs a `SELECT * FROM Employees` query, and writes the results back as an HTML table. `web.xml` maps the servlet class to the `/DatabaseServlet` URL pattern.

Technology Stack

- Apache Tomcat 11 (Jakarta Servlet 6.0, package `jakarta.servlet.*`)
- MySQL Server, queried via the `mysql-connector-j` JDBC driver
- Plain `HttpServlet` (no Spring) — deployed as an exploded WAR directory under Tomcat's `webapps/`

Directory Structure

```
DatabaseServlet/  
├── index.html  
├── src/com/ssn/servlet/DatabaseServlet.java  
└── WEB-INF/  
    ├── web.xml  
    ├── classes/com/ssn/servlet/DatabaseServlet.class  
    └── lib/mysql-connector-j.jar
```

Request Flow

```
Browser (GET form submit)  
  |  
  v  
index.html --submits GET to--> /DatabaseServlet/DatabaseServlet  
  |  
  v  
Tomcat looks up <url-pattern> in web.xml  
  |  
  v  
DatabaseServlet.doGet()  
  |  
  v  
JDBC (com.mysql.cj.jdbc.Driver) -> MySQL "testdb.Employees"  
  |
```

v
ResultSet rows written back as an HTML <table>

Database Setup

```
CREATE DATABASE IF NOT EXISTS testdb;
USE testdb;
CREATE TABLE IF NOT EXISTS Employees (
  id INT, age INT, first VARCHAR(20), last VARCHAR(20)
);
INSERT INTO Employees (id, age, first, last) VALUES
  (101, 20, 'Zara', 'Ali'),
  (102, 24, 'Arjun', 'Kumar'),
  (103, 22, 'Priya', 'Sharma');
```

2. Source Code

File: index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Employee Records</title>
</head>
<body>
  <h1>View Employee Records</h1>

  <!-- method="get" makes the browser send a GET request to DatabaseServlet -->
  <form action="DatabaseServlet" method="get">
    <button type="submit">Show Records</button>
  </form>
</body>
</html>
```

File: web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee https://
jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
  version="6.0">

  <!-- Maps the class to a logical name, then maps that name to a URL. -->
  <servlet>
```

```

        <servlet-name>DatabaseServlet</servlet-name>
        <servlet-class>com.ssn.servlet.DatabaseServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>DatabaseServlet</servlet-name>
        <url-pattern>/DatabaseServlet</url-pattern>
    </servlet-mapping>

</web-app>

```

File: DatabaseServlet.java

```

package com.ssn.servlet;

import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

// Flow: browser submits the GET form in index.html -> Tomcat routes the
// request here based on the <url-pattern> in web.xml -> doGet() opens a
// JDBC connection to MySQL, runs the query, and writes the results as HTML.
public class DatabaseServlet extends HttpServlet {

    private static final String DB_URL = "jdbc:mysql://localhost:3306/testdb";
    private static final String DB_USER = "root";
    private static final String DB_PASSWORD = "";

    protected void doGet(HttpServletRequest request, HttpServletResponse
        response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html><body>");
        out.println("<h1>Employee Records</h1>");
        out.println("<table border='1' cellpadding='6'>");
        out.println("<tr><th>ID</th><th>Age</th><th>First Name</th><th>Last
            Name</th></tr>");

        try {
            Class.forName("com.mysql.cj.jdbc.Driver"); // registers the MySQL
                JDBC driver

```

```

    } catch (ClassNotFoundException e) {
        out.println("</table><p>Driver not found: " + e.getMessage() + "</p></body></html>");
        return;
    }

    try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
        DB_PASSWORD);
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery("SELECT * FROM Employees")) {

        while (rs.next()) {
            out.println("<tr>"
                + "<td>" + rs.getInt("id") + "</td>"
                + "<td>" + rs.getInt("age") + "</td>"
                + "<td>" + rs.getString("first") + "</td>"
                + "<td>" + rs.getString("last") + "</td>"
                + "</tr>");
        }
    } catch (Exception e) {
        out.println("</table>");
        out.println("<p>Database error: " + e.getMessage() + "</p>");
        out.println("</body></html>");
        return;
    }

    out.println("</table>");
    out.println("</body></html>");
}
}

```

Compilation and Deployment Commands

```

# Compile against Tomcat's servlet-api.jar
javac -cp "$CATALINA_HOME/lib/servlet-api.jar" \
    -d WEB-INF/classes src/com/ssn/servlet/DatabaseServlet.java

# Deploy the exploded app directory into Tomcat's webapps folder
cp -r DatabaseServlet $CATALINA_HOME/webapps/

# Start Tomcat
$CATALINA_HOME/bin/catalina.sh run

```

3. Output Screenshots

a. index.html – the GET form

View Employee Records

Show Records

Employee records form

b. After clicking “Show Records”

Employee Records			
ID	Age	First Name	Last Name
101	20	Zara	Ali
102	24	Arjun	Kumar
103	22	Priya	Sharma

Employee records table

Clicking **Show Records** submits a GET request to `/DatabaseServlet/DatabaseServlet`. Tomcat dispatches it to `DatabaseServlet.doGet()` per the mapping in `web.xml`, which queries MySQL and renders the three seeded rows (Zara Ali, Arjun Kumar, Priya Sharma) as an HTML table.

4. Learning Outcomes

This exercise built a full raw-Servlet flow without any framework: an HTML form, a `web.xml` URL mapping, and a compiled `HttpServlet` class talking to MySQL over JDBC, deployed as an exploded directory inside Tomcat's `webapps/`. One real debugging lesson came up while wiring the JDBC driver: even after adding `mysql-connector-j.jar` to `WEB-INF/lib` and calling `Class.forName("com.mysql.cj.jdbc.Driver")`, the very first request after a context reload transiently threw *"No suitable driver found"* — a timing/registration race during Tomcat's webapp classloader reload — which resolved itself on the next request once the driver's static initializer had fully registered with `DriverManager`. This reinforced how `web.xml`'s `<servlet-mapping>` connects a URL pattern to a specific class, and how Tomcat's per-webapp classloader isolation affects JDBC driver registration.